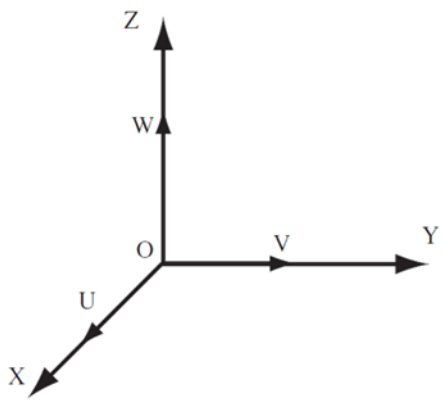
EXAMEN PARCIAL ROBÓTICA 2025-1

Solo envíe los archivos Python.

1. [3 puntos] Sean los sistemas de referencia fijo XYZ y móvil UVW, inicialmente con el mismo origen. Calcular la matriz de rotación que describe la secuencia de rotaciones indicada:
   i. Rotación de 80º alrededor del eje OX
   ii. Rotación de 35º alrededor del eje OZ
   iii. Rotación de 15º alrededor del eje OU
   iv. Rotación de 30º alrededor del eje OY

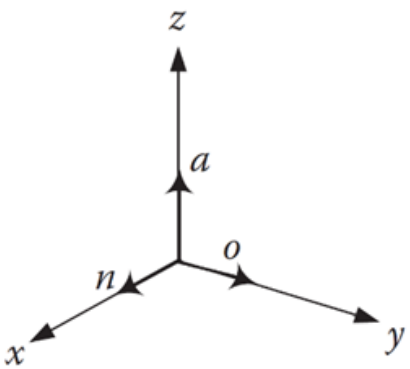


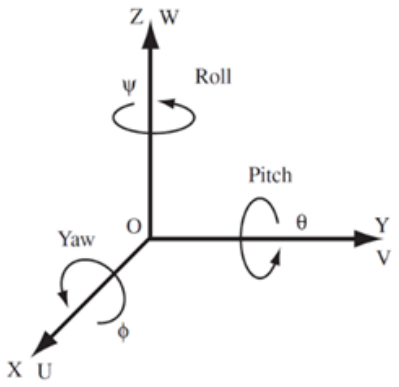
$$R_{\text{tot}} = \begin{bmatrix} 0.709406 & 0.541390 & 0.451264 \\ 0.573576 & -0.071394 & -0.816035 \\ -0.409576 & 0.837735 & -0.361176 \end{bmatrix}$$

2. [3 puntos] Dado el vector **v** = (2, −1, 3), utiliza cuaterniones para calcular el vector **ω** que se obtiene al aplicar una rotación de ángulo θ = π/2 alrededor del eje **y** sobre **v**.

$$\boldsymbol{\omega} = (3, -1, -2)$$

3. [6 puntos] Tenemos un sistema fijo **XYZ** y un sistema móvil **noa** coincidentes. El sistema **noa** se desplazó a lo largo de su propio **eje n** una distancia de 5 unidades, luego se rotó alrededor de su **eje o** un ángulo de 60°, seguido de una rotación de 60° alrededor del **eje z**, luego se trasladó a lo largo de su **eje a** por 3 unidades, y finalmente se roto 45° alrededor del **eje x**.
   ¿Qué ángulos de rotación (Roll, Pitch y Yaw) y qué valores de traslación en coordenadas cartesianas (en los ejes X, Y y Z) deberíamos aplicar para lograr la misma posición y orientación del sistema **noa** resultante?.

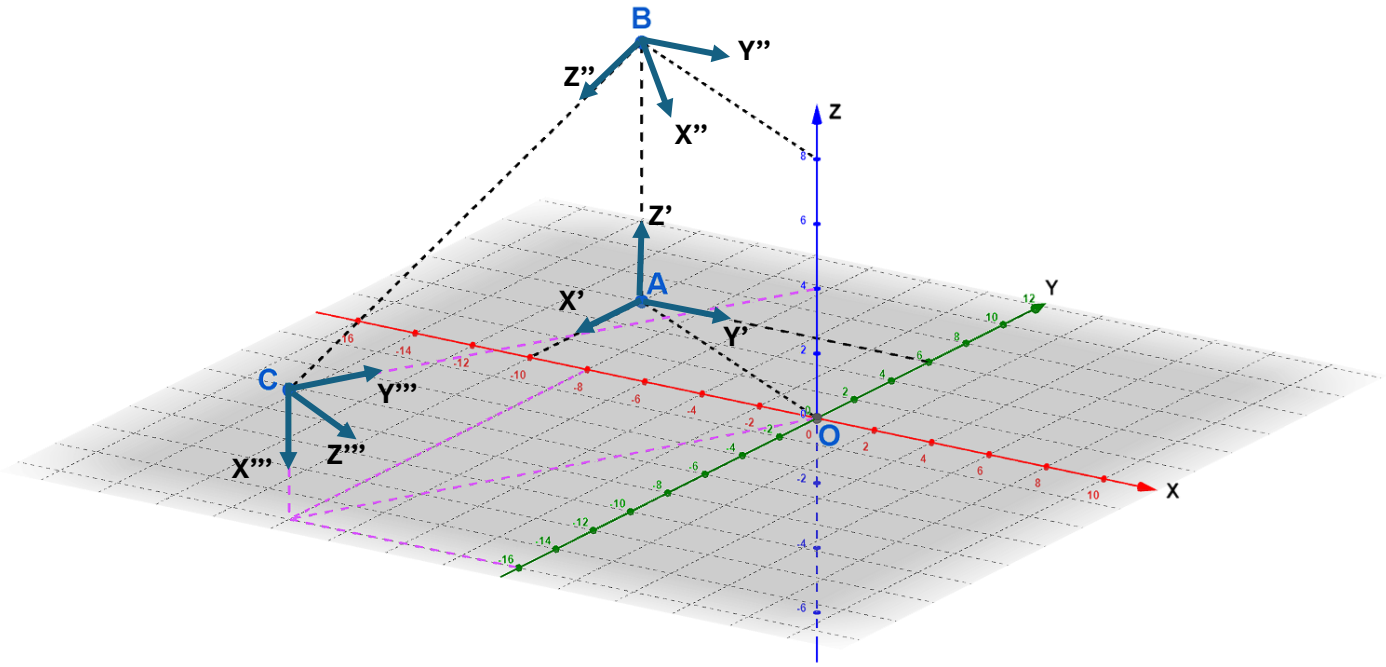




4. Las coordenadas de los puntos A, B y C son respectivamente (-10, 6, 0), (-10, 6, 8) y (-8, -16, 4) respecto al origen O.

   Obtenga la matriz homogénea del sistema CX’’’Y’’’Z’’’ partiendo del sistema OXYZ, pasando primero por AX’Y’Z’ y luego por BX’’Y’’Z’’.

   a) [4 puntos] Usando sistemas fijos.
   b) [4 puntos] Usando sistemas móviles.




```
[[  0.      0.4472   0.8944  -8.    ]
 [ -0.      0.8944  -0.4472 -16.    ]
 [ -1.      0.       0.       4.    ]
 [  0.      0.       0.       1.    ]]
```

SOLUCIÓN 3

$$^{U}T_{B} = Rot(x,45)Rot(z,60)Trans(5,0,0)Rot(o,60)Trans(0,0,3)$$

$$B_{new} = \begin{bmatrix} 0.25 & -0.866 & 0.433 & 3.8 \\ 0.918 & 0.354 & 0.177 & 3.59 \\ -0.306 & 0.354 & 0.884 & 5.71 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

For Cartesian coordinates: $p_x = 3.8, \quad p_y = 3.59, \quad p_z = 5.71$

For RPY angles:

$\phi_a = ATAN2(n_y, n_x) = ATAN2(0.918, 0.25) = 74.8^\circ$

or $\phi_a = ATAN2(-n_y, -n_x) = 254.8^\circ$

$\phi_o = ATAN2(-n_z, (n_x C\phi_a + n_y S\phi_a)) = ATAN2(0.306, 0.951) = 17.8^\circ$

$\phi_o = ATAN2(0.306, -0.951) = 162.2^\circ$

$\phi_n = ATAN2((-a_y C\phi_a + a_x S\phi_a), (o_y C\phi_a - o_x S\phi_a))$

$= ATAN2[0.372, 0.928] = 21.8^\circ$

$\phi_n = ATAN2(-0.372, -0.928) = 201.8^\circ$

$$q_{fin} = (0.6124, -0.3536, 0.6124, 0.3536)$$

$$p' \approx (0, 0.634, 3.098, -2)$$

$$\theta \approx 0.1745 \text{ radianes } (\approx 10^\circ)$$

################ transformaciones.py ######################

```
import numpy as np
def Rz(theta):
  theta=np.radians(theta)
  s=np.sin(theta)
  c=np.cos(theta)
  return np.array([
    [c,-s,0,0],
```

```
        [s,c,0,0],
        [0,0,1,0],
        [0,0,0,1]
        ])
def Rx(theta):
    theta=np.radians(theta)
    s=np.sin(theta)
    c=np.cos(theta)
    return np.array([
        [1,0,0,0],
        [0,c,-s,0],
        [0,s,c,0],
        [0,0,0,1]
        ])
def Ry(theta):
    theta=np.radians(theta)
    s=np.sin(theta)
    c=np.cos(theta)
    return np.array([
        [c,0,s,0],
        [0,1,0,0],
        [-s,0,c,0],
        [0,0,0,1]
        ])
def T(tx,ty,tz):
    return np.array([
        [1,0,0,tx],
        [0,1,0,ty],
        [0,0,1,tz],
        [0,0,0,1]
        ])
def Rk(k,theta):
    theta=np.radians(theta)
    k = k/np.linalg.norm(k)
```

```
    kx, ky, kz = k
    s=np.sin(theta)
    c=np.cos(theta)
    v=1-c
    return np.array([
      [c+kx*kx*v, kx*ky*v-kz*s, kx*kz*v+ky*s,0],
      [ky*kx*v+kz*s, c+ky*ky*v, ky*kz*v-kx*s,0],
      [kz*kx*v-ky*s, kz*ky*v+kx*s, c+kz*kz*v,0],
      [0,0,0,1]
      ])
    ################## transformacionesSymb.py ################
    import sympy as sp

def Rz_symb(angle):
    s=sp.sin(angle)
    c=sp.cos(angle)
    return sp.Matrix([
      [c,-s,0,0],
      [s,c,0,0],
      [0,0,1,0],
      [0,0,0,1]
      ])
def Rx_symb(angle):
    s=sp.sin(angle)
    c=sp.cos(angle)
    return sp.Matrix([
      [1,0,0,0],
      [0,c,-s,0],
      [0,s,c,0],
      [0,0,0,1]
      ])
def Ry_symb(angle):
    s=sp.sin(angle)
    c=sp.cos(angle)
```

```
    return sp.Matrix([
        [c,0,s,0],
        [0,1,0,0],
        [-s,0,c,0],
        [0,0,0,1]
        ])
def T_symb(tx, ty, tz):
    return sp.Matrix([
        [1,0,0,tx],
        [0,1,0,ty],
        [0,0,1,tz],
        [0,0,0,1]
        ])
def Rk_symb(k, angle):
    k = k/sp.sqrt(k.dot(k))
    kx, ky, kz = k
    s=sp.sin(angle)
    c=sp.cos(angle)
    v=1-c
    return sp.Matrix([
        [c+kx*kx*v, kx*ky*v-kz*s, kx*kz*v+ky*s,0],
        [ky*kx*v+kz*s, c+ky*ky*v, ky*kz*v-kx*s,0],
        [kz*kx*v-ky*s, kz*ky*v+kx*s, c+kz*kz*v,0],
        [0,0,0,1]
        ])

################ Ejercicio 3 ##########################
import numpy as np
import sympy as sp
from transformaciones import *
from transformacionesSymb import *

#Traslación 5 unidades a lo largo de n = (1,0,0)
H1 = T(5, 0, 0)
```

```
#Rotación 60 grados alrededor de o = (0,1,0)
H2 = Ry(60)
#Rotación 60 grados alrededor z
H3 = Rz(60)
#Traslación 3 unidades a lo largo de a = (0,0,1)
H4 = T(0, 0, 3)
#Rotación 60 grados alrededor de x
H5 = Rx(45)

H_res = H5 @ (H3 @ (H1 @ H2)) @ H4
print('Matriz de transformación resultante')
print(H_res, '\n')

phi, theta, psi = sp.symbols('phi theta psi')
R_roll_pitch_yaw = Rz_symb(psi)@Ry_symb(theta)@Rx_symb(phi)
print('Matriz correspondiente a la transformación Roll Pitch Yaw')
sp.pprint(R_roll_pitch_yaw)
print()

ang_theta = np.degrees(np.arcsin(-H_res[2,0]))
ang_psi = np.degrees(np.arcsin(H_res[1,0]/np.cos(np.radians(ang_theta))))
ang_phi = np.degrees(np.arcsin(H_res[2,1]/np.cos(np.radians(ang_theta))))

v_trasl = np.array([H_res[0,3], H_res[1,3], H_res[2,3]])

print('Ángulo de rotación Yaw en grados:', ang_phi)
print('Ángulo de rotación Pitch en grados:', ang_theta)
print('Ángulo de rotación Roll en grados:', ang_psi)
print('Vector de traslación:', v_trasl, '\n')

H_roll_pitch_yaw_traslacion = T(v_trasl[0],v_trasl[1],v_trasl[2]) @ Rz(ang_psi) @ Ry(ang_theta) @ Rx(ang_phi)
print('Comprobación de la transformación:')
print(H_roll_pitch_yaw_traslacion)
```